

SpecEE: Accelerating Large Language Model Inference with Speculative Early Exiting

Jiaming Xu
Shanghai Jiao Tong University
Shanghai, China
jiamingxu@sjtu.edu.cn

Jiayi Pan
Shanghai Jiao Tong University
Shanghai, China
pan_jiayi@sjtu.edu.cn

Yongkang Zhou
Shanghai Jiao Tong University
Shanghai, China
zeenny.williams@sjtu.edu.cn

Siming Chen
Shanghai Jiao Tong University
Shanghai, China
320220941351@lzu.edu.cn

Jinhao Li
Shanghai Jiao Tong University
Shanghai, China
kimholee@sjtu.edu.cn

Yaoxiu Lian
Shanghai Jiao Tong University
Shanghai, China
lianyaoxiu@sjtu.edu.cn

Junyi Wu
Shanghai Jiao Tong University
Shanghai, China
kimi_wu@sjtu.edu.cn

Guohao Dai*
Shanghai Jiao Tong University;
Infinigence-AI; SII
Shanghai, China
daiguohao@sjtu.edu.cn

Abstract

Early exiting has recently emerged as a promising technique for accelerating large language models (LLMs) by effectively reducing the hardware computation and memory access. In this paper, we identify that the LLM vocabulary serves as the runtime search space of the early exiting predictor and significantly influences the predictor workload (e.g., $\sim 20\%$ overall inference latency with $\sim 3 \times 10^4$ vocabulary size in Llama2). We propose a novel paradigm using speculative models to reduce this search space, while addressing three critical challenges for further predictor optimization. (1) **Time-consuming predictor with high computational complexity.** Current predictor designs leverage basic models with high-dimensional input that ignore inherent data variation and GPU parallelization opportunities, resulting in $\sim 15\%$ overall inference latency. (2) **Under-utilization of layer-wise predictor deployment.** Current early exiting systems treat the predictor in each layer equally without considering the activation frequencies of layer-wise predictors, leading to $\sim 20\%$ inference overhead. (3) **Exponential mapping complexity of predictor in speculative decoding.** Each token in the token tree of speculative decoding is treated as an independent search space when applying the current early exiting mapping, leading to exponential mapping complexity and failing to incorporate the high-throughput benefits

To address the above challenges, we present *SpecEE*, a fast LLM inference engine with speculative early exiting. (1) **At the algorithm level**, we propose the **speculation-based lightweight predictor design** by exploiting the probabilistic correlation between the speculative tokens and the correct results and high parallelism of GPUs. (2) **At the system level**, we point out that not all layers need a predictor and design the **two-level heuristic predictor scheduling engine** based on skewed distribution and contextual similarity. (3) **At the mapping level**, we point out that different decoding methods share the same essential characteristics, and propose the **context-aware merged mapping for predictor** with efficient GPU implementations to support speculative decoding, and form a framework for various existing orthogonal acceleration techniques (e.g., quantization and sparse activation) on cloud and personal computer (PC) scenarios, **successfully pushing the Pareto frontier of accuracy and speedup**. It is worth noting that *SpecEE* can be applied to any LLM by negligible training overhead in advance without affecting the model's original parameters. Extensive experiments show that *SpecEE* achieves $2.25\times$ and $2.43\times$ speedup with Llama2-7B on cloud and PC scenarios respectively. The code is open-sourced in <https://github.com/infinigence/SpecEE>

CCS Concepts

- Computing methodologies → Machine learning approaches;
- Computer systems organization → Real-time systems.

Keywords

Large Language Model, Machine Learning and System, GPU

ACM Reference Format:

Jiaming Xu, Jiayi Pan, Yongkang Zhou, Siming Chen, Jinhao Li, Yaoxiu Lian, Junyi Wu, and Guohao Dai. 2025. SpecEE: Accelerating Large Language Model Inference with Speculative Early Exiting. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*, June 21–25, 2025, Tokyo, Japan. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3695053.3730996>

*Corresponding Author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ISCA '25, Tokyo, Japan

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1261-6/25/06
<https://doi.org/10.1145/3695053.3730996>

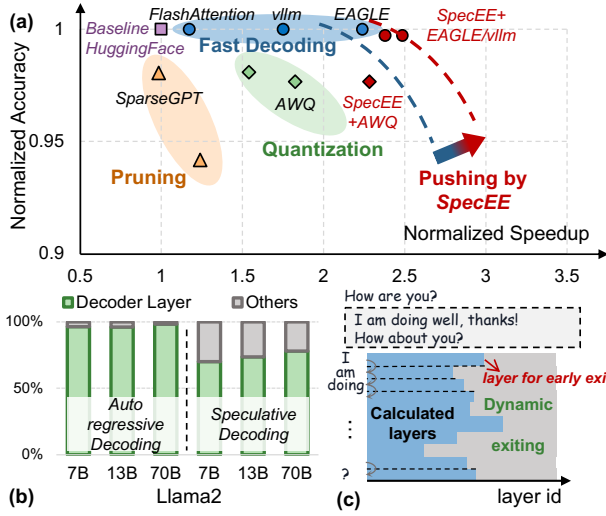


Figure 1: (a) Pareto frontier of accuracy and speedup towards LLM inference and deployment. The detailed normalized accuracy and speedup are obtained with Llama2-7B on an NVIDIA RTX 4090 GPU. (b) The ratio of the time of the decoder layer to end-to-end inference time in the original LLM. The data of two decodings is obtained based on Hugging Face [42] and EAGLE [27] frameworks. (c) Different numbers of decoder layers are needed for different token generation.

1 Introduction

Towards the advancement of Artificial General Intelligence (AGI), generative large language models (LLMs) have been successfully applied across various domains, significantly enabling the rapid development of numerous downstream tasks (e.g., agent application [43], code generation [21] and robotics [8]). Driven by the scaling law, more and more LLMs with an increasing number of parameters (e.g., Grok-1 [44] with 314B parameters) have proven remarkable performance in many scenarios. However, this further results in significant memory requirements and inference latency, which poses great challenges for the deployment of practical applications. For cloud service vendors of LLMs, the extended response time translates to increased infrastructure costs (e.g., energy) and suboptimal user experiences. For example, it is estimated that OpenAI consumes 260.42 MWh of energy per day [48], which translates into a cost of \$26,042 per day, based on U.S. industrial electricity prices of about 10 cents per kWh. This is approximately five times the average monthly income of \$4,831 in the United States [3].

Consequently, many previous works have explored techniques to accelerate LLM inference and reduce infrastructure cost for deployment, encompassing algorithm optimization, system enhancements, and hardware advancements [26, 47]. Some of these works (e.g., fast decoding [1, 7, 18, 23, 27]) ensure the consistency of results, while others (e.g., pruning and quantization [10, 25, 28]) may lead to accuracy loss, thus forming a Pareto frontier of accuracy and speedup towards LLM inference and deployment as shown in Figure 1(a). However, due to the lack of consideration of the relationship between the dynamic input and the static model in these works, the

multiple cascaded layers in the original model account for 70 ~ 95% of end-to-end inference shown in Figure 1(b), becoming the primary bottleneck for pushing the Pareto frontier forward.

The inference of LLM is to generate the token with the highest probability from the full vocabulary through cascading decoder layers, which is essentially an online search problem and the search space is the full vocabulary. Early exiting algorithm is an emerging optimization in dynamic neural networks [15, 24] that aims to timely and efficiently predict when search termination occurs. Several recent works [9, 19] have highlighted that not all decoder layers are necessary during inference in LLMs, enabling dynamic adjustments for different tokens. They suggest that the LLM parameters should be adjusted based on the complexity of the task during inference. As shown in Figure 1(c), during token generation, different tokens require different forward layers to be generated. Commonly, these works entail integrating data-driven predictors (e.g., Support Vector Machine (SVM) [16] and Multilayer Perceptron (MLP) [36]) after each layer and structuring relevant features as input information to predict exiting.

In this paper, we point out that the LLM vocabulary also serves as the online search space (the linear operation with the $hidden_dim \times vocabulary_size$ weight, called *LM Head*, in LLM) of the early exiting predictors and significantly influences the workload (e.g., ~ 20% inference overhead with $\sim 3 \times 10^4$ vocabulary size of Llama2 [41] in AdaInfer [9]). Therefore, we propose a **novel paradigm using speculative models to reduce this search space by generating speculative tokens, successfully achieving $10^4 \times$ search space reduction for predictors** shown in Figure 2(b). To apply the insight for further predictor optimization, the following challenges remain unsolved.

Challenge-1: Time-consuming predictor with high design complexity. Current LLM early exiting predictor [9, 19] commonly need to traverse the full search space (multiplied with the complete *LM Head*) to get the relevant data before prediction, and then take the raw high-dimensional ($> 4 \times 10^3$) data as input for prediction without feature analysis and extraction. To accommodate high-dimensional input data, the predictor adopts a basic model (e.g., SVM in AdaInfer [9]) with high computational complexity without considering the parallelism of GPUs, resulting in ~ 30% overall computation and ~ 15% overall inference latency.

Challenge-2: Under-utilization of layer-wise predictor deployment. Current early exiting system equally treat the decoder layers of LLMs and deploy the predictor after each layer. Statistical data indicates that the success probability of the predictors follows a skewed distribution, meaning that early exiting typically occurs at a fixed set of layers for different tokens. This implies that the computations of most other predictors are ineffective in the majority of cases, resulting in ~ 20% additional inference overhead.

Challenge-3: Exponential mapping complexity of predictor in speculative decoding. Speculative decoding [2, 4, 27] proposes the pattern of draft generation and token verification through tree-based token structure to address the poor throughput of autoregressive decoding. However, when applying the current early exiting mapping which aims to associate the tokens with the search space of predictors, each token of the token tree is treated as an

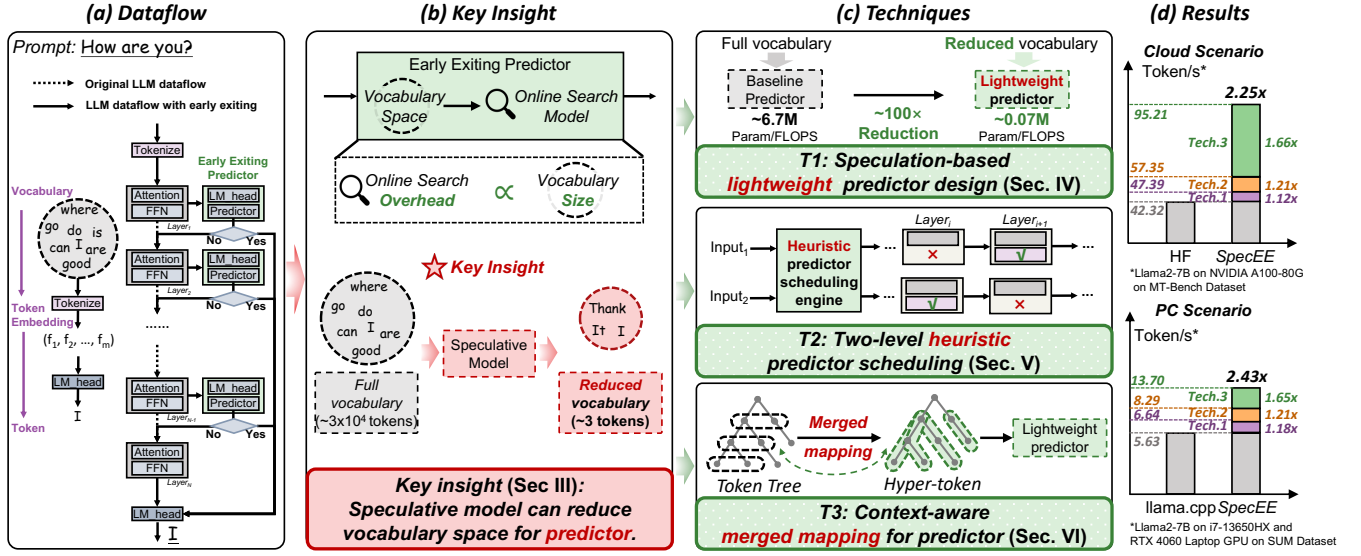


Figure 2: Overview of *SpecEE*. (a) Dataflow of early exiting. (b) Key insight: Speculative model can reduce vocabulary space for predictor. (c) Techniques on predictor optimization from Section 4 to Section 6. (d) Results on cloud and PC scenarios.

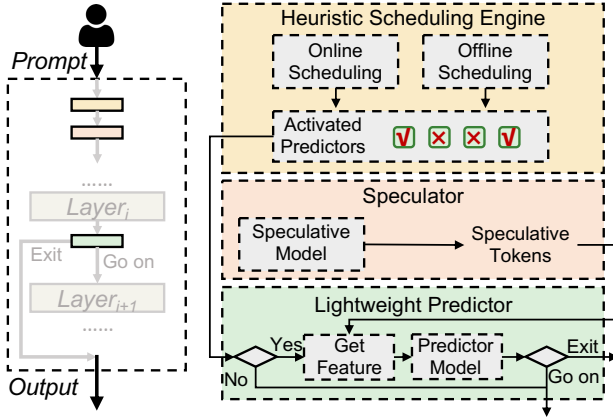


Figure 3: Architecture of *SpecEE*.

independent search space without considering the contextual semantics, leading to the exponential mapping complexity and the failure of incorporating the high-throughput benefits.

To address the above challenges, we present *SpecEE*, a fast LLM inference engine with speculative early exiting. The techniques of *SpecEE* can be summarized into three levels as follows.

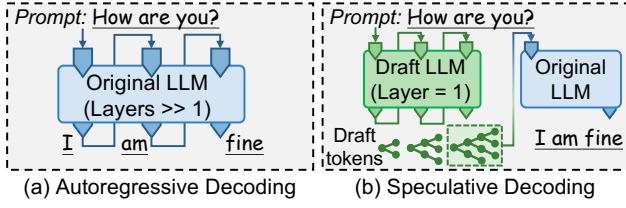
(1) Speculation-based lightweight predictor design at the algorithm level. Based on the key insight mentioned above, we point out that the probability shift of speculative tokens is strongly correlated with whether it is the correct result and extract the several meaningful metrics as prediction features. To fully leverage the parallelism of GPUs, We adopt the lightweight MLP as predictor, achieving $\sim 100\times$ parameters and FLOPS reduction and $\sim 1.12\times$ end-to-end inference acceleration shown in Figure 2(c)-T1 and (d).

(2) Two-level heuristic predictor scheduling at the system level. We further point out that not all layers require predictor integration and computation based on statistical results, and propose

the two-level heuristic predictor scheduling, which contains offline scheduling and online scheduling to achieve heuristic control over predictor integration and computation during the inference shown in Figure 2(c)-T2. Offline scheduling allocates predictors based on skewed distribution on offline activation frequency from extensive statistical analysis. Online scheduling is performed runtime based on the contextual similarity of the exit layer positions, where the probability that the exit layer position of the current token is within ± 2 layers of the previous five tokens exceeds 70%. The two-level heuristic scheduling achieves $\sim 68\%$ predictor reduction and $\sim 1.21\times$ inference acceleration shown in Figure 2(d).

(3) Context-aware merged mapping for predictors at the mapping level. Based on the contextual similarity in the exit layer positions mentioned in Technique (2), we point out that this property also applies to the tree-based speculative decoding, where contextual dependencies exist between the input token tree. Therefore, we propose the context-aware merged mapping for predictors with efficient GPU implementations supporting speculative decoding, which merges each path in the tree-based tokens into a *hyper-token* shown in Figure 2(c)-T3, turning exponential mapping complexity into linear complexity and achieving $1.66\times$ inference acceleration shown in Figure 2(d). Moreover, due to the orthogonality, *SpecEE* also forms a framework for various existing orthogonal acceleration techniques (e.g., quantization [28] and sparse activation [38]) on cloud and PC scenarios, successfully pushing the Pareto frontier of accuracy and speedup shown in Figure 1(a).

The architecture of *SpecEE* is shown in Figure 3. After obtaining the input prompt, the heuristic scheduling engine comprising offline and online scheduling mechanisms is employed to identify the predictors that require activation. Subsequently, the speculative model is invoked to generate speculative tokens. Between each pair of consecutive decoder layers, if the predictor should be activated,

**Figure 4: Two decoding methods of LLM.**

features are retrieved and the predictor model is utilized to decide whether to go on with the inference process or to exit.

We implement *SpecEE* on the NVIDIA Tesla A100 80GB and RTX 4090 24GB GPUs for cloud scenario and Lenovo Legion Y7000 with i7-13650HX CPU and NVIDIA RTX 4060 Laptop 8GB GPU for PC scenario. As illustrated in Figure 2(d), extensive experiments results on several LLMs (Llama2 [41]) show that *SpecEE* achieves up to $2.25\times$ and $2.43\times$ speedup compared with implementation by Hugging Face on cloud scenario and llama.cpp on PC scenario with all the techniques with negligible accuracy loss. Notably, *SpecEE* can be applied to any LLM by negligible training overhead in advance without affecting the model’s original parameters.

2 Background

2.1 Large Language Models

During token generation of the LLM inference, the traditional autoregressive decoding approach generates one token at a time based on input prompts and previously generated tokens, as shown in Figure 4(a). This ensures context - dependency for natural language processing (NLP) tasks during computation. In the attention mechanism of transformer backbone in LLMs, the generation of each token only considers the preceding content and is independent of future tokens. To reduce redundant computations, existing LLM inference systems use *kv_cache* to store keys and values of previous content. Given the self-attention mechanism’s inability to handle nonlinear relationships, the Feed-Forward Network (FFN) is introduced to capture deeper, abstract features, which compensate for the limitations of self-attention mechanism.

2.2 Speculative Decoding in LLMs

Autoregressive decoding generates a single token based on the input tokens during inference, resulting in poor throughput. Speculative decoding is proposed to address the limitation. As illustrated in Figure 4(b), it uses a smaller speculative draft language model (DLM) to generate speculative tokens autoregressively, forming tree-structured tokens. These tokens are then verified by the target language model (TLM) to decide which path to accept. This enables TLM to generate multiple tokens in one forward computation, achieving inference acceleration. DLM is crucial in end-to-end inference as the quality of its output determines if effective acceleration can be achieved. As DLM has fewer parameters, methods like joint training and knowledge distillation (e.g., Medusa [2], EAGLE [27]) are used to align its performance with TLM.

The primitive DLM of speculative decoding often uses a smaller-scale model with the same structure. However, it’s hard to find the smallest such DLM, and different types of models can’t serve as DLM for each other, limiting its versatility. LOOKAHEAD [12]

Table 1: Related Works on Skip Layer and Early Exiting.

	Memory	Prediction	Training	Latency
AdaInfer [9]	Low	Heavy	Low	High
RAEE [19]	High	Heavy	Low	High
MoD [35]	Low	Light	High	Low
D-LLM [45]	Low	Light	High	Low
<i>SpecEE</i>	Low	Light	Low	Low

,MEDUSA [2] and EAGLE [27] are highly efficient speculative decoding methods that have achieved substantial acceleration effects. However, the end-to-end time consumption of the TLM in these methods accounts for a relatively high proportion, making the TLM the main bottleneck for performance.

2.3 Skip Layer and Early Exiting

Several recent studies [9, 19, 35, 45] have successfully explored the applicability of early exiting and skip layer in LLM inference. However, as shown in Table 1, existing early exiting algorithms [9, 19] introduce the prediction process with significant additional overhead in the decoding process, resulting in inefficient end-to-end inference. While existing skip layer algorithms [35, 45] have achieved promising performance in end-to-end inference, they require pre-training or fine-tuning of the LLM, which requires a significant cost in terms of hardware and training time.

Skip Layer. The Mixture-of-Depths (MoD) [35] method uses a router to let some tokens bypass blocks, and D-LLM [45] places a dynamic decision module before each transformer layer. However, both MoD and D-LLM have limitations in terms of training overhead. They rely on training to learn routing or dynamic mechanisms, consuming a lot of resources and time. They often need retraining for different tasks and datasets, increasing application complexity and cost and possibly affecting their deployment and performance.

Early Exiting. AdaInfer [9] points out three specific features that serve as good indicators for early exiting during LLM inference. However, fetching these features needs to integrate LM head after each layer which results in deal time consumption. RAEE [19] constructs an early exiting information database. It retrieves early exiting data based on embedding similarity and calculates the early exiting layer by probability superposition. However, its database construction is highly complex, and the inherent retrieval time leads to suboptimal end-to-end performance.

As is shown in Table 1, existing early exiting methods usually have a heavy prediction phase and high end-to-end latency, while current skip layer methods always incur high training overhead. Therefore, we aim to propose an approach that features low memory usage, light prediction, low training cost, and low latency.

3 Motivation

3.1 Key Challenges of Early Exiting

As mentioned in Section 2.3, AdaInfer [9] requires traversing the full vocabulary (e.g., $\sim 3 \times 10^4$ tokens in Llama2 [41]) during prediction to obtain the probabilities of all tokens as predictor features, while RAEE [19] requires searching the pre-built database (with a size exceeding several gigabytes) related to vocabulary, resulting in $> 30\%$ overall computation and $\sim 20\%$ end-to-end inference latency.

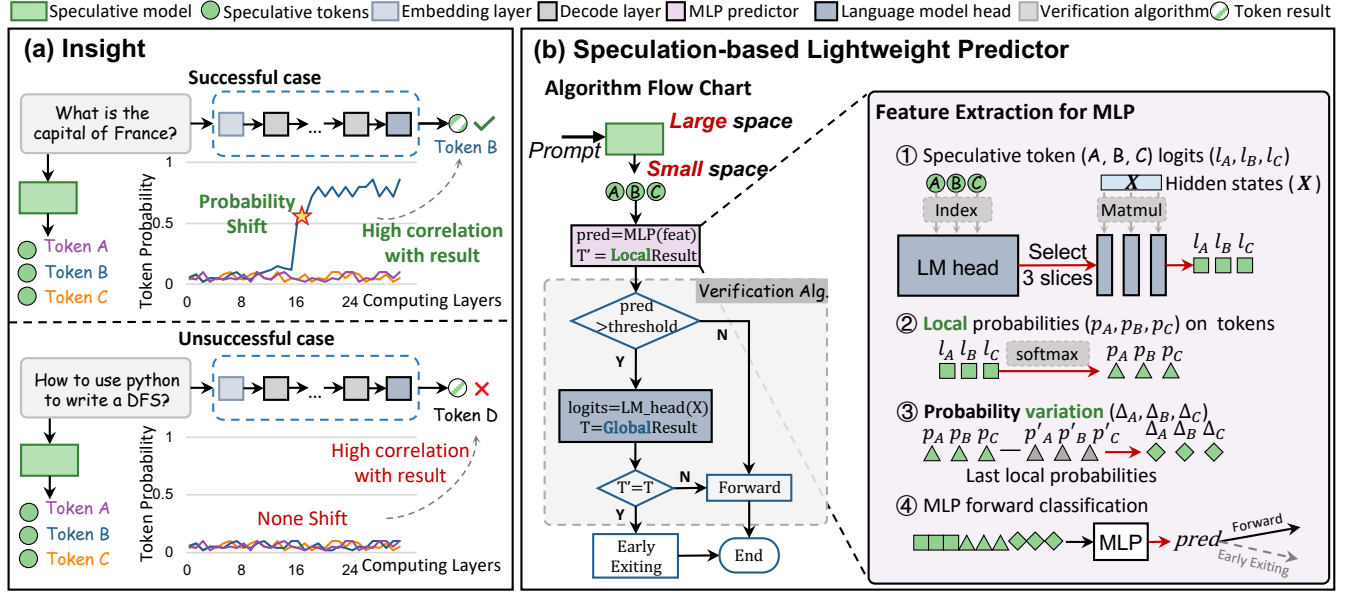


Figure 5: (a) The insight on probability shift detailed in Section 4.2. (b) The algorithm flow chart and the feature extraction in speculation-based vocabulary space reduction.

We analyze the online search process of the predictor and find that its overhead is primarily results from the traversal of the vocabulary, making the computational cost positively correlated with the size of the vocabulary shown in Figure 2(b). Consequently, we identify that the vocabulary also serves as the search space for the early exiting predictor, which inherently contributes to the overhead. Therefore, we consider that the key challenge of early exiting is **how to reduce the vocabulary space using low-cost methods** involving low memory, light prediction, negligible training shown in Table 1 to finally enable effective online token prediction and low end-to-end inference latency.

3.2 Key Insight

Inspired by speculation in computer system design and speculative decoding detailed in Section 2.2, we consider that the role of DLM in speculative decoding is to generate speculative tokens for TLM. From the perspective of TLM, the output from DLM provides a potential way to streamline the range of token selection (*i.e.*, search space), even if the actual output may not always fall within this range. Furthermore, as mentioned in Section 2.2, the goal of training DLM is to ensure that the results of TLM align as closely as possible with these speculative tokens. In other words, with a strong enough DLM, it is possible to fully limit the results of the TLM to the range of speculative tokens (*i.e.*, valid small space in the insight of Figure 2(a)).

Therefore, we propose a **novel paradigm using speculative models to reduce the search space** shown in Figure 2(b). The data in EAGLE [27] shows that it only requires ~ 3% memory and inference overhead of original LLM and ~ 48 hours on RTX 3090 training overhead, which also matches our requirements in Table 1.

(a) Speculative tokens:	project	team	company	your	yourself	my
Speculative token logits:	0.29	0.37	0.58	2.29	3.37	2.98
Local probabilities:	0.29	0.32	0.39	0.18	0.58	0.24
Probability gap:	0.05	0.12	0.11	0.05	0.12	0.11
Correct Decision:	Proceeding			Exiting		

(b) Speculative tokens:	your	yourself	my	to	your	on
Speculative token logits:	2.29	3.37	2.98	8.63	9.80	8.92
Local probabilities:	0.18	0.58	0.24	0.17	0.58	0.24
Probability gap:	0.05	0.12	0.11	0.05	0.12	0.11
Correct Decision:	Exiting			Proceeding		

Figure 6: Analysis on feature selection. It is necessary to select three all features for prediction to prevent misjudgment.

4 Speculation-based Lightweight Predictor

4.1 Motivation: Time-consuming Predictor

Though the search space can be effectively reduced by the speculative model, the design of current LLM early exiting predictor [9, 19]) still relies on directly utilizing high-dimensional raw data (*e.g.*, $\sim 5 \times 10^3$ in Llama2-7B) retrieved from the search space as input features, without performing any feature analysis or extraction. As illustrated in Figure 2(c)-T1, this raw high-dimensional data imposes significant demands on the predictor internal design, requiring complex architectures with a large number of parameters and computational overhead to effectively capture the implicit information contained within these high-dimensional features. Moreover, current predictor designs adopt traditional basic models (*e.g.*, SVM in AdaInfer [9]) for intuitiveness and interpretability, ignoring the parallel computing opportunities provided by GPUs.

4.2 Insight: Probability Shift

We need to explore the feasibility of the new paradigm utilizing the speculative tokens generated by the speculative model as the

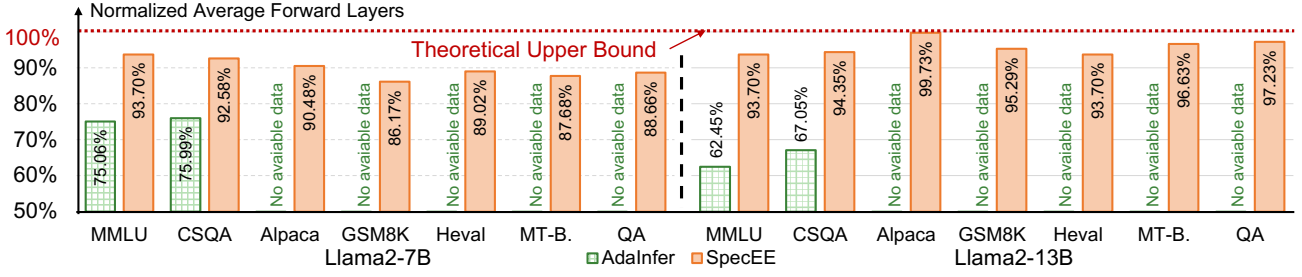


Figure 7: The gap between actual average forward layers and theoretical average forward layers of AdaInfer and SpecEE. AdaInfer only provides the average forward layers of MMLU and CommonsenseQA in the datasets in Section 7.1.3.

reduced search space. As illustrated in Figure 5(a), we conduct experiments on the probability variation of tokens in the reduced space and point out that during LLM inference, if the final result token is within the reduced space, the probability of this token tends to rise sharply at a certain layer, while the probabilities of other tokens remain stable at lower values. Conversely, if the final output is not in the streamlined space, the probabilities of all tokens in the streamlined space tend to remain stable at lower values. We refer to this phenomenon as the **probability shift**.

4.3 Approach: Lightweight Design

Based on the insight and analysis mentioned above, we design the speculation-based lightweight predictor. The predictor design includes three parts, feature selection, judgment mechanism and correction algorithm.

4.3.1 Feature Selection. We selected speculative token logits, local probabilities, probability variation as the input features for the predictor in each layer. Below is a detailed description of each feature and the rationale behind our selection.

(1) **Speculative token logits** are the result of the matrix multiplication ($1 \times \text{hidden_dim} \times \text{num_speculatives}$) between the output of each layer (i.e., *hidden_states*) and the *speculative_lm_head* which refers to the columns of the *lm_head* corresponding to the speculative tokens, providing direct insight into the confidence of LLM on speculative tokens.

(2) **Local probabilities** are the result of applying the softmax function to speculative token logits. The probabilities are based on local information rather than global information, reflecting the likelihood of speculative tokens within the streamlined search space.

(3) **Probability variation** is the difference between the local probabilities in the current layer and the last layer, capturing changes in the probability across layers.

Our analysis has indicated that the probability variation of tokens is a crucial factor for prediction and we select **probability variation** as a feature. However, as illustrated in Figure 6(a), we observe that the variation of 0.12 can result from either 0.32 – 0.20 or 0.58 – 0.46. The predictor in Figure 6(a) shouldn't allow exiting in the left while the exit probability should be higher in the right. Therefore, we consider using probability variation alone as feature is insufficient and introduce **local probabilities** as an additional feature. Moreover, the local probability may be the same when speculative token logits are different shown in Figure 6(b). In such case,

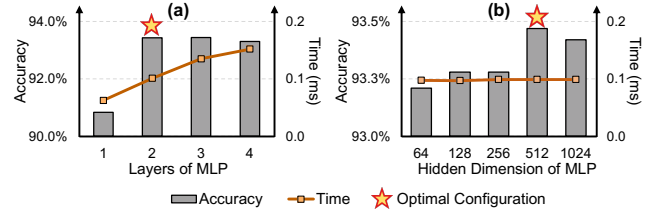


Figure 8: Design space exploration on the predictor configuration. (a) The accuracy and execution time of the predictor with changing layers and controlled hidden dimension (512). (b) The accuracy and execution time of the predictor with changing hidden dimensions and controlled layers (2).

the predictor in the right conversely makes a proceeding decision and thus we further take **speculative token logits** as a feature.

4.3.2 Judgment Mechanism. Based on the features mentioned above, we configure the speculative model to generate 4 speculative tokens each time, resulting in the feature dimension of 12 (4×3). To fully leverage the high computational capacity of the GPU's Tensor Cores, we employ a two-layer MLP as the predictor with the hidden dimension of 512 instead of traditional machine learning methods (e.g., SVM). The predictor employs the ReLU activation function and sets a Sigmoid function at the output layer to handle the binary classification task. The features are fed into the predictor, and the decision to exit is determined by comparing the predictor's output to a predefined threshold (i.e., 0.5).

4.3.3 Verification Algorithm. As described in Section 4.3.1, the local probabilities are derived from local information rather than global information. To verify the prediction results, we further propose the verification algorithm by incorporating global information. As illustrated in Figure 5, we compute global token logits using the full *lm_head* and check if the token with the highest global logits is present in the speculative tokens. If it is, we exit and output that token, and if not, the model proceeds to the next layer.

Example. Figure 9 shows an example of the speculation-based predictor computation. We use "How are you?" as the prompt and take the ending at layer 22 of LLM as an example. The speculative tokens are firstly generated based on the prompt, forming the speculative LM Head. Feature extraction from the hidden states is followed during the LLM inference for the prediction. Finally, the

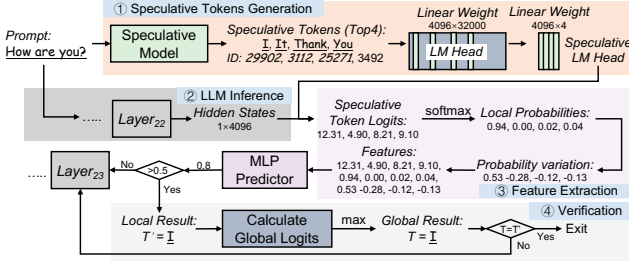


Figure 9: Example of predictor computation.

verification is performed by comparing the global token from LM Head with the local token.

Design Space Exploration. To reduce the predictor’s execution time while maintaining accuracy, we focus on the number of layers and the hidden dimension. We explore the design space through using the control-variable approach shown in Figure 8. The accuracy represents the predictor’s performance on the test set detailed in Section 7.4.4. The optimal configuration is a 2-layer MLP with the hidden dimension of 512, which is our final configuration.

4.4 Evaluation

The most ideal scenario for the acceleration based on early exiting is that the actual average exit layer of the method approaches the theoretical average earliest exit layer. Thus, we evaluate the average exit layer obtained by our method and the theoretical exit layer of each dataset. As illustrated in Figure 7, our method is closer to the theoretical value than AdaInfer, which is the only work about the early exiting of Llama2 models. Our method maintains close alignment with theoretical values across different datasets, exhibiting strong stability. This proximity to the theoretical exit layers is also a key reason why our approach maintains accuracy without degradation shown in Section 7.4.1.

5 Two-level Heuristic Scheduling Engine

5.1 Motivation

Based on the speculation-based lightweight predictor proposed in Section 4, we conducted experiments on a series of datasets in Section 7.1.3 using Llama2-7B [41]. However, the end-to-end acceleration was not significant, showing only an average speedup of about 15%. Despite this, the average number of executed layers was around 23, suggesting that the theoretical acceleration ratio could reach approximately $\sim 33\%$ ($32/(23 + 1)$). The overhead of the speculative model is roughly equivalent to the execution time of a single decoder layer. Therefore, we believe that it is the overall overhead of the predictors that slows down the end-to-end inference. The predictor overhead is defined as $T \times L$, where T is the execution time of a single predictor and L is the number of layers integrated with the predictor (e.g., $L = 32$ in Llama2-7B).

Figure 8 illustrates the relationship between the time overhead, accuracy, and parameter configurations of the predictor, and the final experiment is conducted with the optimal configuration (2 layers MLP with 512 hidden dimension). Thus, reducing the overall predictor overhead can only be achieved by decreasing L . Moreover, we point out that the sum of the probabilities of all layers with exit

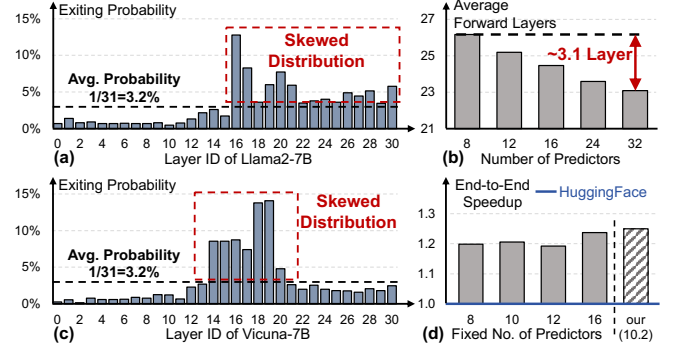


Figure 10: (a)(c) The statistical exiting probability on the 31 (0 ~ 30) layers in Llama2-7B and Vicuna-7B (no predictor needed for last layer). (b) The average forward layers on fixed predictors with random positions in Llama2-7B. Random positions of predictors lead to up to ~ 3.1 layers gap. (d) The end-to-end speedup on different fixed numbers of predictors and our dynamic predictor numbers in Llama2-7B.

probabilities falling within the bottom 50% does not exceed 20% shown in Figure 10(a) and (c), which implies that prediction in these layers are mostly unnecessary. However, Figure 10(b) indicates that blindly reducing L can hinder timely exiting, leading to an increase (~ 3.1 layers) in the average number of executed layers and inference latency. Therefore, we consider that the key issue is **how to accurately control the quantity (L) and position of predictors to achieve end-to-end inference acceleration.**

5.2 Insights and Analysis: Skewed Distribution and Context Similarity

Inspired by the dynamic resource allocation in system optimization [11, 20, 34], we consider that it is necessary to dynamically adjust the number and position of predictors according to the actual situation, focusing on two key variables during LLM inference, model selection and context input.

Skewed Distribution. We investigated the distribution of predictor results across two models shown in Figure 10(a) and (c), and identified a skewed distribution with about 50% of the layers where the statistical probability of exiting is less than the average probability 3.2%. This skewness also varies across different models.

Context Similarity. Additionally, inspired by the context similarity observed in language processing [30] and sparse activation [29, 31], we focus on the relationship of the exit layer of the current token and the last few tokens as shown in Figure 11. Statistical results show that the exit layer of the current token has $\sim 80\%$ probability of being near (e.g., ± 2 layer) the exit layers of the last 5 tokens. Experiments reveal that the set consisting of the exit layers of the last 5 tokens and their neighboring layers amounts to approximately 10.2 layers on average, as shown in Figure 10(d). Based on average probability calculations, the probability that the exit layer of a token falls within this set should be approximately 31.8%. However, experiments indicate that this probability is as high as 80%. Thus, we can conclude that there is a significant context similarity in the location of the exit layer.

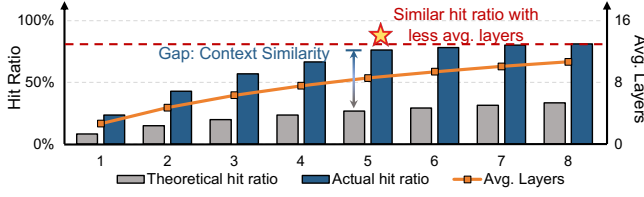


Figure 11: The explanation for context similarity. The hit ratio of the current token’s exit layer within the vicinity (± 2 layers) of the exit layers of the last N tokens (x -axis), as well as the average number of layers after taking the union of last N tokens’ exit layers and neighboring layers.

5.3 Approach: Two-level Heuristic Scheduling

Based on the above insight and analysis mentioned above, we propose the two-level adaptive scheduling. The approach includes two parts, offline scheduling and online scheduling shown in Figure 12.

Offline Scheduling. Given that different LLMs exhibit variations in exit probability distributions shown in Figure 10(a) and (c), offline scheduling is employed to collect data offline for the LLM. It performs inference on the LLM with all predictors fully integrated using numerous prompts, collecting data from each predictor and ranking them by frequency. The result is integrated into the model as a model configuration parameter which is model-dependent and only needs to be executed offline once for a LLM.

Online Scheduling. Based on the context similarity mentioned above, during the inference, we always maintain a circular queue of length N , representing the local context attention span (e.g., 5 tokens mentioned above). Additionally, we use an array with a length equal to the total number of layers (L). The circular queue sequentially records the exit layer positions for the last N tokens, while the i -th element of the array tracks the number of times the i -th layer has been near (e.g., ± 2 and itself) the exit layers of last N tokens recorded in the circular queue.

Finally, the quantity and position of predictors are determined by the union of a subset of results selected by the offline scheduling, and the results from the online scheduling. The performance gap between the fixed number of predictors and the dynamic number of predictors is shown in Figure 10(d). The dynamic selection in *SpecEE*

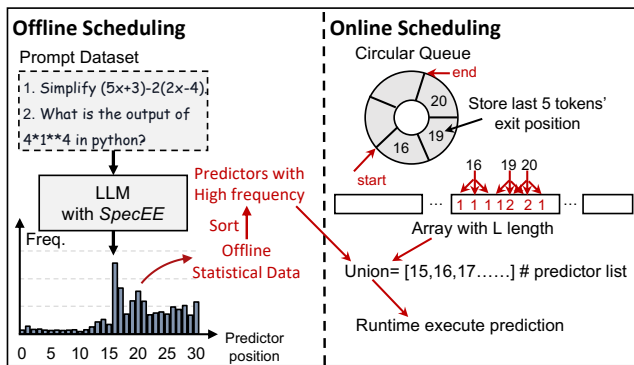


Figure 12: Dataflow of heuristic scheduling.

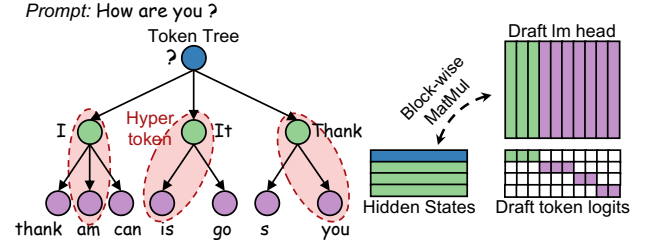


Figure 13: The hyper-token for speculative decoding and the customized GPU implementation developed based on cutlass [33] and MegaBlocks [13] for calculating the draft token logits in *SpecEE* for speculative decoding.

achieves the highest end-to-end speedup with fewer predictors (only ~ 10.2 layers).

6 Context-aware Merged Mapping for Predictor

6.1 Motivation

Speculative decoding successfully achieves high throughput through the pattern of draft generation and token verification. As illustrated in Figure 13, the token tree is composed of multiple tokens at each level by autoregressive generation of the speculative model. The first generation is three green tokens (Top3 probability) based on the prompt. And then these three tokens will be concatenated and fed into the speculative model and get the purple tokens at next level. All the tokens will be concatenated and fed into the target LLM to verify the tokens through one forward computation.

When applying the early exiting during verification inference, the current mapping for predictors treats each token in the token tree as an independent search space without considering the contextual semantics. For example, the root token (?) and its three speculative tokens (*I*, *It*, *Thank*) are mapped a predictor to decide the early exiting, while the green token (*I*) and its 3 speculative tokens (*thank*, *am*, *can*) are also mapped a predictor at the same time. Moreover, these predictors are independent of each other, which means the overall mapping complexity is the product of the complexities of individual predictors, resulting in an exponential complexity. Therefore, we consider that the key issue is **how to design a novel mapping for speculative decoding that maintains low complexity.**

6.2 Approach: Context-aware Merged Mapping

Algorithm. We analyze the nature of the speculative decoding and point out that early exiting shares a common essence across both decoding methods. In autoregressive decoding, early exiting is used to predict the next token based on the current token, while in speculative decoding, it is used to predict a token sequence within the token tree. However, according to the fundamental principle of early exiting, the exit position of a token sequence should be determined by the rearmost position of the exiting layers within it, reflecting an obvious Cannikin law that significantly impacts end-to-end performance. For example, if the token *I* exit at the 22nd layer while the token *am* exit at the 30th layer, the exiting position of the token path (*I*, *am*) is 30th layer. Inspired by the

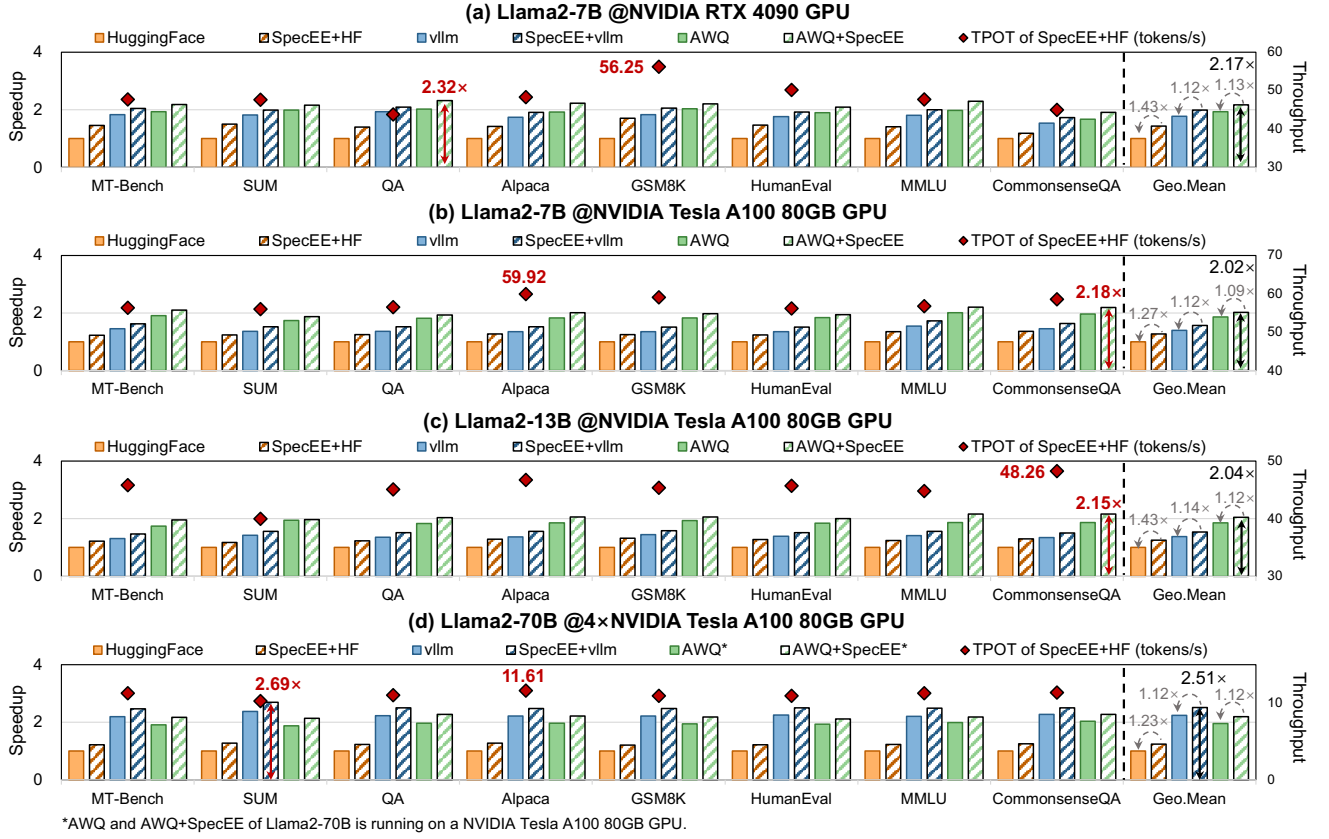


Figure 14: The speedup and throughput of Llama2-7B, Llama2-13B and Llama2-70B on NVIDIA RTX 4090 GPU and Tesla A100 80GB GPU for autoregressive decoding in cloud scenario.

context similarity in Section 5.2, we highlight that tokens within a token path share contextual relationships, achieving centralized exit positions and alleviating the performance loss due to Cannikin law. Thus, we propose the context-aware merge-based mapping for predictors in speculative decoding, where the tokens in a path of the token tree is merged as a single hyper-token as shown in Figure 13. This abstraction allows the early exiting in speculative decoding to be addressed similarly to autoregressive decoding.

Implementation. To efficiently compute the features of the hyper-token in Section 4.3.1 and minimize the additional overhead caused by early exiting, designed a custom GPU operator implementation shown in Figure 13 inspired the block-wise general matrix multiplication in MegaBlocks [13] based on the group GEMM implementation of cutlass [33].

6.3 Extension: Support for Orthogonal Acceleration Techniques

SpecEE is a dataflow developed initially based on autoregressive decoding, and it is entirely orthogonal to the techniques mentioned in Figure 1(a). Therefore, we have selected the following mainstream techniques in the cloud and PC scenarios for integration, successfully pushing the Pareto frontier forward. The performance is the red labels in Figure 1(a), and the detailed results are in Section 7. The detailed implementation is as follows.

Fast Decoding. We select the Paged Attention in vllm [23] as the representative of fast decoding in the cloud scenario and implement the speculation-aided early exiting dataflow aligned with vllm including the PageAttention usage for DLM.

Quantization. We select the AWQ [28] with official implementation as the representative of quantization for cloud scenario and integrate the original DLM (not quantized model) for candidate token generation to achieve the early exiting dataflow.

Sparsification. We select the Powerinfer [38] with sparse activation as the representative of sparsification for PC scenario and develop the C++ code based on the llama.cpp and the official implementation of Powerinfer, achieving the GPU-CPU hybrid inference.

7 Evaluation

7.1 Experimental Setup

We evaluate the performance of *SpecEE* with various LLMs on two scenarios, cloud scenario and PC scenario. We compare the performance with several mainstream LLM inference engines in these two scenarios.

7.1.1 Hardware Platforms. We evaluate the performance of *SpecEE* and other LLM engines on the platforms to make a comprehensive comparison. We choose two different GPUs for cloud scenarios, NVIDIA Tesla A100-80GB GPU and NVIDIA RTX 4090 24GB GPU. For the PC scenario, we select the Lenovo Legion Y7000 PC with

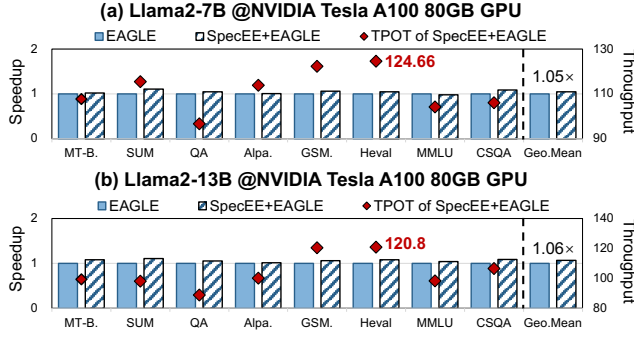


Figure 15: (a) The speedup and throughput of Llama2-7B and Llama2-13B on NVIDIA Tesla A100 80GB GPU for speculative decoding in cloud scenario.

NVIDIA RTX 4060 Laptop GPU (8GB) and Intel i7-13650HX CPU. We show the detailed configuration in Table 2.

Table 2: Hardware Platforms

	Cloud Senario		PC Senario
NVIDIA GPU	Tesla A100 80GB	RTX 4090 24GB	RTX 4060 Laptop 8GB
	CUDA 12.1	CUDA 12.1	CUDA 12.6
CPU	Intel Xeon Platinum 8358	AMD EPYC 7542	13th Gen Intel Core i7-13650HX
	2.60GHz	2.90GHz	2.6GHz

7.1.2 LLM Engine Baselines. For the cloud scenario, we implement *SpecEE* using the Pytorch-based front-end with the C++ and CUDA backend for NVIDIA GPUs. We integrated *SpecEE* into Hugging Face [42], vllm [23] and AWQ [28] and thus we compare the inference performance in decoding speedup and throughput with the above frameworks on two GPUs mentioned above.

For the PC scenario, we implement *SpecEE* using the llama.cpp framework with the C++ and CUDA backend for NVIDIA GPUs and Intel CPUs. We integrated *SpecEE* into llama.cpp [14] and PowerInfer [38] and thus we compare the inference performance in decoding speedup and throughput with these two frameworks on the PC mentioned above. The maximum number of new tokens generated per inference is 256.

Table 3: Model Configuration

Model	Dimension	Heads	Layers	Context Length
Llama2-7B	4096	32	32	4k
Llama2-13B	5120	40	40	4k
Llama2-70B	8192	64	80	4k

7.1.3 Models and Datasets. We evaluate the performance of *SpecEE* with other LLM inference engines on the chat models of Llama2-7/13/70B [41]. Table 3 shows the configuration of these models.

For evaluation on speedup and throughput, we select nine datasets in real scenarios: MT-Bench (MT-B.) [46], SUM [32],

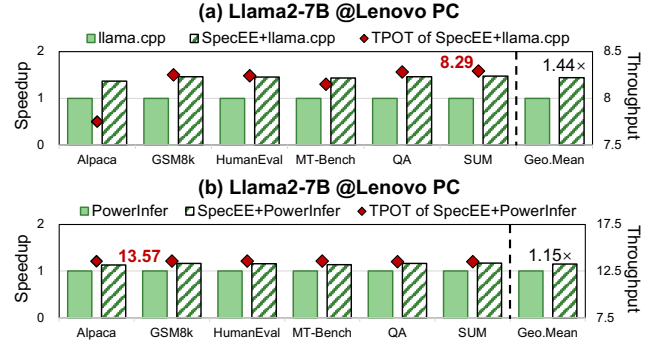


Figure 16: (a) The speedup and throughput of Llama2-7B on Lenovo PC compared with llama.cpp and PowerInfer.

QA [22], Alpaca [40], GSM8K [6], HumanEval (Heval) [5], MMLU [17], CommonsenseQA (CSQA) [39], and SST2 [37]. We select seven representative datasets: MMLU, CommonsenseQA, SST2 and GSM8k for accuracy evaluation, and SUM, MT-Bench and Alpaca for perplexity (PPL) evaluation. We followed the commonly used few-shot settings in the LLM community. These tasks cover question answering, code generation, sentiment analysis, and text generation.

7.2 Evaluation on Speedup and Throughput

7.2.1 Cloud Scenario. We divide the evaluation on speedup and throughput into two parts for autoregressive decoding and speculative decoding.

For the autoregressive decoding, we compare the performance after the integration of *SpecEE* and Hugging Face [42], vllm [23] and AWQ [28] with the original performance of Hugging Face, vllm and AWQ using Llama2-7B, Llama2-13B and Llama2-70B on NVIDIA GPUs with 8 datasets in Section 7.1.3. As illustrated in Figure 14, *SpecEE* achieves the average 1.43 $\times$, 1.12 $\times$ and 1.13 $\times$ speedup on Llama2-7B compared with Hugging Face, vllm and AWQ on RTX 4090 respectively, and achieves average 1.27 $\times$, 1.12 $\times$ and 1.09 $\times$ speedup compared with Hugging Face, vllm and AWQ on Tesla A100. And the average speedup on Tesla A100 over Hugging Face, vllm and AWQ is 1.43 $\times$, 1.14 $\times$ and 1.12 $\times$ for Llama2-13B and 1.23 $\times$, 1.12 $\times$ and 1.12 $\times$ for Llama2-70B.

For the speculative decoding, we compare the performance after the integration of *SpecEE* and EAGLE [27] with the original performance of EAGLE on Tesla A100 80GB GPU using Llama2-7B and Llama2-13B. As illustrated in Figure 15, *SpecEE* achieves the average 1.05 $\times$ and 1.06 $\times$ speedup compared with EAGLE on Llama2-7B and Llama2-13B respectively. Overall, *SpecEE* achieves average 2.25 $\times$ speedup compared with HuggingFace on Llama2-7B.

7.2.2 PC Scenario. For the PC scenario, we compare the performance after the integration of *SpecEE* and llama.cpp [14], PowerInfer [38] with the original performance of llama.cpp and PowerInfer using Llama2-7B on Lenovo PC. As illustrated in Figure 16, *SpecEE* achieves the average 1.25 $\times$ and 1.15 $\times$ speedup compared with llama.cpp and PowerInfer respectively.

Table 4: Accuracy Evaluation

Task	MMLU		CommonSenseQA		SST		GSM8k		SUM		MT-Bench		Alpaca	
	Acc. ↑	#Avg. L ↓	Acc. ↑	#Avg. L ↓	Acc. ↑	#Avg. L ↓	Acc. ↑	#Avg. L ↓	PPL ↓	#Avg. L ↓	PPL ↓	#Avg. L ↓	PPL ↓	#Avg. L ↓
Llama2-7B (32 Layers)														
Dense	45.30	32	61.43	32	86.24	32	20.62	32	10.09	32	6.49	32	6.86	32
AdaInfer	43.73	28.91	53.00	27.90	-	-	0.00 [*]	-	-	-	-	-	-	-
SpecEE	44.64	23.16	61.26	22.90	85.89	23.55	20.00	23.13	10.69	23.79	8.44	23.22	6.32	21.96
AWQ	44.61	32	58.31	32	84.98	32	23.16	32	7.95	32	5.80	32	10.01	32
AWQ+SpecEE	44.45	23.27	59.05	22.94	84.98	22.81	22.11	23.22	8.08	23.50	5.34	23.19	5.38	22.28
Llama2-13B (40 Layers)														
Dense	53.58	40	67.57	40	93.00	40	33.87	40	8.76	40	6.64	40	4.93	40
AdaInfer	52.44	36.35	62.48	34.60	-	-	-	-	-	-	-	-	-	-
SpecEE	53.37	24.93	67.16	24.59	92.78	25.92	33.58	26.34	7.23	27.80	7.76	26.02	4.82	24.96
AWQ	49.70	40	64.95	40	91.74	40	28.42	40	6.53	40	4.66	40	5.81	40
AWQ+SpecEE	50.43	28.15	65.85	26.57	91.40	27.62	27.32	28.15	7.66	28.27	4.00	27.22	6.08	26.34
Llama2-70B (80 Layers)														
Dense	60.74	80	76.82	80	94.27	80	55.79	80	5.88	80	4.25	80	2.44	80
SpecEE	60.54	53.25	76.74	52.14	94.04	49.40	55.79	56.51	6.07	57.58	3.85	55.31	1.94	52.88
AWQ	59.53	80	71.72	80	94.15	80	55.05	80	5.87	80	4.72	80	2.42	80
AWQ+SpecEE	60.17	50.14	76.58	53.79	94.15	49.26	55.08	56.34	6.63	56.78	4.93	52.48	2.55	53.87

^{*} This data is from D-LLM [45].

7.3 Hardware Evaluation

7.3.1 Energy Efficiency. We compare the power of dense model and *SpecEE* with the Llama2-7B on NVIDIA A100 GPU (TDP 400W) using the MT-Bench dataset. We monitor GPU power consumption changes by `nvidia-smi` provided by NVIDIA during inference, and the statistic data shows that *SpecEE* can reduce the average power from 201W to 182W, achieving $\sim 10\%$ power reduction and $\sim 1.57\times$ energy efficiency. We consider this is primarily because the predictor in *SpecEE* is a memory-bound operator, and its workload is lower compared to other modules of LLM, resulting in underutilized computational resources.

7.3.2 Hardware Insight. We further profile the power and latency of the lightweight predictor on the NVIDIA A100 and Lenovo PC with RTX 4060 Laptop. *SpecEE* exhibits similar latency on both the A100 and Lenovo PC but more power consumption on A100 ($\sim 142W$ vs $\sim 85W$). The A100 is an integrated training-inference architecture, while the Lenovo PC is mostly for inference. When GPUs like the A100 are used for LLM inference, power consumption should also be considered. Our advice is that future integrated training-inference GPUs could adopt a big-little core design, similar to ARM SoCs, selectively activating only a portion of CUDA cores or other computing resources to optimize power efficiency.

7.4 Overhead Evaluation

7.4.1 Accuracy Loss. We evaluate the accuracy of the models in Section 7.1.3 with the seven datasets. We followed the commonly used few-shot settings in the LLM community. Table 4 shows that *SpecEE* achieves negligible accuracy loss ($< 1\%$) compared with the original model and far outperforms the AdaInfer, whose data is obtained in its paper, both on accuracy and average forward layers.

7.4.2 Memory Usage. Due to the draft language model and predictors, the memory usage of *SpecEE* is initially higher than original model. As illustrated in Figure 17, the memory usage is about 0.9GB and 1.4GB more than the original model. In this paper, we choose the open-source DLM of EAGLE [27] as the speculative model. This speculative model is the main contributor to the initial additional memory usage. As mentioned in Section 5.1, the predictor is an MLP

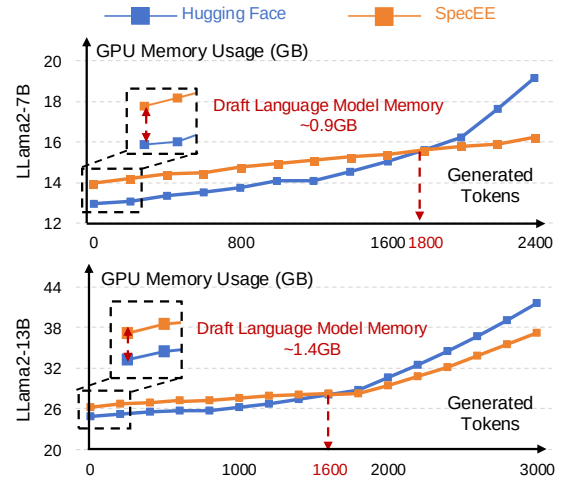


Figure 17: GPU memory usage of Llama2-7B and Llama2-13B with token generation.

of 2 layers with 512 hidden dimension. Thus the total memory usage of all the predictors is about 416KB ($((12 \times 512 + 512 \times 1) \times 32 \times 4 / 1024)$ in Llama2-7B with 4 draft tokens. Compared to the DLM, the memory usage of predictors can be negligible.

7.4.3 Speculative Model Training. The model training overhead of *SpecEE* is very low compared to the works on the skip layer described in Section 2.3. *SpecEE* only needs to train a speculative model and the predictors. In this paper, we select the DLM provided by EAGLE [27] as the speculative model. As described in the paper of EAGLE, the speculative model for Llama2-7B only needs 24 hours of training using an RTX 3090 GPU.

7.4.4 Predictor Offline Training and Runtime Overhead. We use the MT-Bench dataset as the prompt for inference and obtain the intermediate layer (e.g., 0 $\sim$ 30 in Llama2-7B without the last layer) features as training data, along with the token generated by early exiting at the intermediate layer. This token is compared with the token generated after all layers, and the label is set to True if they match, or False otherwise. We totally get about 16K training data

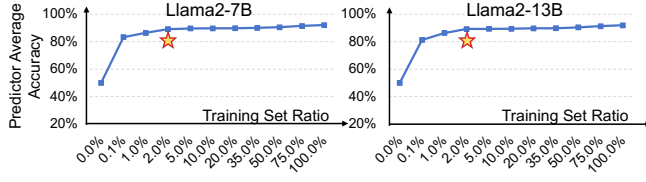


Figure 18: Predictor training of Llama2-7B/13B.

for each predictor, which takes about 1 hour on NVIDIA A100 80GB GPU. And then the training of all predictors takes about 10 minutes with total data. Figure 18 shows relationship between the training data size and predictor accuracy. We only need about $\sim 2\%$ training data to achieve good performance, which totally only takes about 5 minutes. We also profile the runtime overhead of predictor in Llama2-7B on a NVIDIA A100 GPU. The inference of *SpecEE* is ~ 0.016 s/token while the overhead of predictors is 0.0009s/token, which is about 5.6% inference latency.

7.5 Ablation Study

We select the Llama2-7B on NVIDIA Tesla A100-80GB for ablation study and select the Hugging Face as the code base and the baseline. The overall results are in Figure 19.

7.5.1 T1: Speculation-based vocabulary space reduction. The speculation-based vocabulary space reduction aims to reduce the vocabulary space through the speculative model. For the Llama2-7B on NVIDIA Tesla A100-80GB GPU, *SpecEE* only achieves about $1.08\times$ speedup across the 8 datasets shown in Figure 19. We have analyzed that the inefficiency is caused by redundant predictor integration and computation in Section 5.1.

7.5.2 T2: Two-level adaptive scheduling for efficient token prediction. Two-level adaptive scheduling is thus proposed for efficient token prediction. Integrated with this technique, *SpecEE* finally achieves an average $1.27\times$ speedup across the 8 datasets for the Llama2-7B on NVIDIA Tesla A100-80GB GPU shown in Figure 19.

7.5.3 T3: Merge-based multi-vocabulary to hyper-token mapping. Figure 19 shows the overall performance of *SpecEE* with techniques 1 to 3, achieving the outstanding performance. For a fair comparison, we have compared the *SpecEE* with EAGLE shown in Figure 15. *SpecEE* achieves about $1.06\times$ compared with EAGLE [27].

8 Conclusion

In this paper, we propose the novel paradigm using the speculative model to reduce the search space, providing a new perspective to consider LLM acceleration. We think the methodology and perspective can be extended to further studies on machine learning architecture and system design considering search space reduction.

We apply the paradigm to the early exiting for LLM acceleration, and present the *SpecEE*, a fast LLM inference engine with speculative early exiting. *SpecEE* proposes three techniques for further predictor optimization at three levels of algorithm, system and mapping, and achieves $2.25\times$ and $2.43\times$ speedup on Llama2-7B in cloud and PC scenarios, respectively, successfully pushing the Pareto frontier of accuracy and speedup.

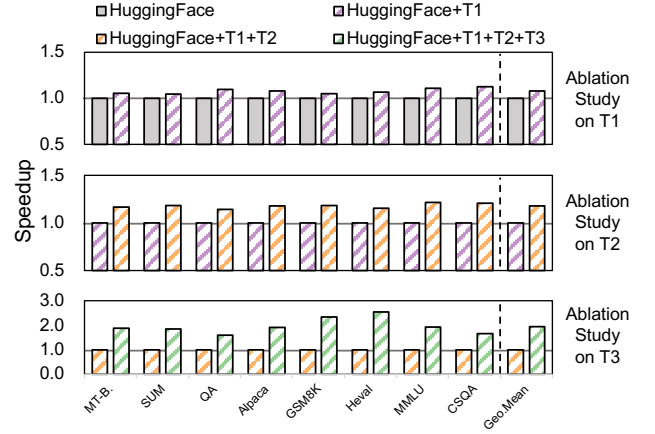


Figure 19: Ablation study of three techniques in *SpecEE*.

Acknowledgments

This work was sponsored by the National Natural Science Foundation of China (No. 62104128, U21B2031), Shanghai Rising-Star Program (No. 24QB2706200) and Beijing Douyin Information Service Co., Ltd.

References

- [1] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, et al. 2022. DeepSpeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.
- [2] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. arXiv:2401.10774 [cs.LG]
- [3] CEIC. 2024. United States Monthly Earnings. [Online]. <https://www.ceicdata.com/en/indicator/united-states/monthly-earnings>.
- [4] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. arXiv preprint arXiv:2302.01318 (2023).
- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgren Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] <https://arxiv.org/abs/2107.03374>
- [6] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168 [cs.LG] <https://arxiv.org/abs/2110.14168>
- [7] Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. arXiv preprint arXiv:2307.08691 (2023).
- [8] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. 2023. PaLM-E: An Embodied Multimodal Language Model. In arXiv preprint arXiv:2303.03378.
- [9] Siqi Fan, Xin Jiang, Xiang Li, Xuying Meng, Peng Han, Shuo Shang, Aixin Sun, Yequan Wang, and Zhongyuan Wang. 2024. Not all layers of llms are necessary during inference. arXiv preprint arXiv:2403.02181 (2024).

- [10] Elias Frantar and Dan Alistarh. 2023. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International Conference on Machine Learning*. PMLR, 10323–10337.
- [11] Tom ZJ Fu, Jianbing Ding, Richard TB Ma, Marianne Winslett, Yin Yang, and Zhenjie Zhang. 2015. DRS: Dynamic resource scheduling for real-time analytics over fast streams. In *2015 IEEE 35th International Conference on Distributed Computing Systems*. IEEE, 411–420.
- [12] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. 2024. Break the Sequential Dependency of LLM Inference Using Lookahead Decoding. arXiv:2402.02057 [cs.LG]
- [13] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. 2022. MegaBlocks: Efficient Sparse Training with Mixture-of-Experts. arXiv:2211.15841 [cs.LG] <https://arxiv.org/abs/2211.15841>
- [14] Georgi Gerganov. 2023. LLM inference in C/C++. [Online]. <https://github.com/ggerganov/llama.cpp>.
- [15] Yizeng Han, Gao Huang, Shiji Song, Le Yang, Honghui Wang, and Yulin Wang. 2021. Dynamic Neural Networks: A Survey. arXiv:2102.04906 [cs.CV] <https://arxiv.org/abs/2102.04906>
- [16] Marti A. Hearst, Susan T Dumais, Edgar Osuna, John Platt, and Bernhard Scholkopf. 1998. Support vector machines. *IEEE Intelligent Systems and their applications* 13, 4 (1998), 18–28.
- [17] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. arXiv:2009.03300 [cs.CY] <https://arxiv.org/abs/2009.03300>
- [18] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Yuhang Dong, Yu Wang, et al. 2024. FlashDecoding++: Faster Large Language Model Inference with Asynchronousization, Flat GEMM Optimization, and Heuristics. *Proceedings of Machine Learning and Systems* 6 (2024), 148–161.
- [19] Lianming Huang, Shangyu Wu, Yufei Cui, Ying Xiong, Xue Liu, Tei-Wei Kuo, Nan Guan, and Chun Jason Xue. 2024. RAEE: A Training-Free Retrieval-Augmented Early Exiting Framework for Efficient Inference. *arXiv preprint arXiv:2405.15198* (2024).
- [20] Qingjia Huang, Kai Shuang, Peng Xu, Jian Li, Xu Liu, and Sen Su. 2014. Prediction-based dynamic resource scheduling for virtualized cloud systems. *Journal of Networks* 9, 2 (2014), 375.
- [21] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A Survey on Large Language Models for Code Generation. *arXiv preprint arXiv:2406.00515* (2024).
- [22] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics* 7 (2019), 453–466.
- [23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [24] Stefanos Laskaridis, Alexandros Kouris, and Nicholas D. Lane. 2021. Adaptive Inference through Early-Exit Networks: Design, Challenges and Directions. In *Proceedings of the 5th International Workshop on Embedded and Mobile Deep Learning (MobiSys '21)*. ACM. doi:10.1145/3469116.3470012
- [25] Jinhao Li, Shiyao Li, Jiaming Xu, Shan Huang, Yaoxiu Lian, Jun Liu, Yu Wang, and Guohao Dai. 2023. Enabling Fast 2-bit LLM on GPUs: Memory Alignment, Sparse Outlier, and Asynchronous Dequantization. *arXiv preprint arXiv:2311.16442* (2023).
- [26] Jinhao Li, Jiaming Xu, Shan Huang, Yonghua Chen, Wen Li, Jun Liu, Yaoxiu Lian, Jiayi Pan, Li Ding, Hao Zhou, Yu Wang, and Guohao Dai. 2024. Large Language Model Inference Acceleration: A Comprehensive Hardware Perspective. arXiv:2410.04466 [cs.AR] <https://arxiv.org/abs/2410.04466>
- [27] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. In *International Conference on Machine Learning*.
- [28] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. arXiv:2306.00978 [cs.CL] <https://arxiv.org/abs/2306.00978>
- [29] Chi Ma, Mincong Huang, Ying Zhang, Chao Wang, Yujie Wang, Lei Yu, Chuan Liu, and Wei Lin. 2024. First Activations Matter: Training-Free Methods for Dynamic Activation in Large Language Models. *arXiv preprint arXiv:2408.11393* (2024).
- [30] George A Miller and Walter G Charles. 1991. Contextual correlates of semantic similarity. *Language and cognitive processes* 6, 1 (1991), 1–28.
- [31] Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. 2023. Relu strikes back: Exploiting activation sparsity in large language models. *arXiv preprint arXiv:2310.04564* (2023).
- [32] Ramesh Nallapati, Bowen Zhou, Caglar Gulcehre, Bing Xiang, et al. 2016. Abstractive text summarization using sequence-to-sequence rnns and beyond. *arXiv preprint arXiv:1602.06023* (2016).
- [33] NVIDIA. 2017. CUTLASS: CUDA Templates for Linear Algebra Subroutines. [Online]. <https://github.com/NVIDIA/cutlass>.
- [34] Yanghua Peng, Yixin Bao, Yangrui Chen, Chuan Wu, and Chuanxiong Guo. 2018. Optimus: an efficient dynamic resource scheduler for deep learning clusters. In *Proceedings of the Thirteenth EuroSys Conference*. 1–14.
- [35] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. 2024. Mixture-of-Depths: Dynamically allocating compute in transformer-based language models. arXiv:2404.02258 [cs.LG]
- [36] Frank Rosenblatt. 1958. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological review* 65, 6 (1958), 386.
- [37] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*. 1631–1642.
- [38] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2023. PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU. arXiv:2312.12456 [cs.LG]
- [39] Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. 2019. CommonsenseQA: A Question Answering Challenge Targeting Commonsense Knowledge. arXiv:1811.00937 [cs.CL] <https://arxiv.org/abs/1811.00937>
- [40] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An Instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca.
- [41] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shriti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [42] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Online, 38–45. <https://www.aclweb.org/anthology/2020.emnlp-demos.6>
- [43] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. 2023. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155* (2023).
- [44] xAI. 2024. Open Release of Grok-1. [Online]. <https://github.com/xai-org/grok-1>.
- [45] yikun jiang, Huanyu Wang, Lei Xie, Hanbin Zhao, Chao Zhang, Hui Qian, and John C.S. Lui. 2024. D-LLM: A Token Adaptive Computing Resource Allocation Strategy for Large Language Models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=UIOjGTHKQG>
- [46] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv:2306.05685 [cs.CL] <https://arxiv.org/abs/2306.05685>
- [47] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, et al. 2024. A survey on efficient inference for large language models. *arXiv preprint arXiv:2404.14294* (2024).
- [48] Zodhya. 2023. Optimizing Inference on Large Language Models with NVIDIA TensorRT-LLM, Now Publicly Available. [Online]. <https://medium.com/@zodhyatech/how-much-energy-does-chatgpt-consume-4c61a17aef85>.

A Artifact Appendix

A.1 Abstract

This appendix provide guidance on reproducing the key results of *SpecEE* on the accuracy and speedup for cloud scenario and PC scenario on Llama2-7B. Our artifacts provide code files on the zenodo website and demonstrate the software dependencies on both scenarios. We have provided a detailed workflow on how to reproduce the key results in Figure 14, Figure 16 and Table 4. The two scenarios require the different hardware platforms, Linux Server with NVIDIA Tesla A100-80GB for cloud scenario and Lenovo Legion Y7000 PC with NVIDIA RTX 4060 Laptop GPU (8GB) and Intel i7-13650HX CPU for PC scenario. Besides, the ReadME.md file in artifacts includes comprehensive guidelines for reproduction and code introduction.

A.2 Artifact check-list (meta-information)

- **Algorithm:** Large Language Models, Early Exiting
- **Program:** Python, Shell, C++
- **Compilation:** NVCC, GCC
- **Model:** Llama2-7B-chat [41], Llama-2-7B-Chat-AWQ [28], EAGLE-llama2-chat-7B [27]
- **Data set:** MT-Bench, SUM, QA, Alpaca, GSM8K, HumanEval in Section 7.1.3
- **Run-time environment:** Conda with Python3 and others in requirement.txt of the provided files
- **Hardware:** Linux Server with NVIDIA Tesla A100-80GB GPUs for Cloud scenario and Lenovo Legion Y7000 PC with NVIDIA RTX 4060 Laptop GPU (8GB) and Intel i7-13650HX CPU.
- **Execution:** Python command and llama.cpp command
- **Metrics:** Speedup and Accuracy
- **Output:** Json files or direct value output
- **Experiments:** All speedup results in Figure 14(b) and all accuracy results of Llama2-7B on Table 4.
- **How much disk space required (approximately)?:** ~100GB
- **How much time is needed to prepare workflow (approximately)?:** 6 hours
- **How much time is needed to complete experiments (approximately)?:** 8 hour
- **Publicly available?:** Yes
- **Code licenses (if publicly available)?:** MIT License
- **Data licenses (if publicly available)?:** MIT License
- **Archived (provide DOI)?:** <https://doi.org/10.5281/zenodo.15102802>

A.3 Description

A.3.1 How to access. The code and other files are publicly available at <https://doi.org/10.5281/zenodo.15102802>. The example commands for artifact evaluation are in the README.md.

A.3.2 Hardware dependencies. The results of *SpecEE* need to be reproduced on two scenarios, cloud scenario and PC scenario. The cloud scenario requires the Linux server with NVIDIA Tesla A100-80GB GPU. The PC scenario requires the Lenovo Legion Y7000 PC with NVIDIA RTX 4060 Laptop GPU (8GB) and Intel i7-13650HX CPU.

A.3.3 Software dependencies. The backend compilation is NVCC for CUDA Code and GCC for C++ Code. The other software dependencies are the Python module environment in Anaconda, which are described in detail in the requirement.txt of the provided code file.

A.3.4 Data sets. We evaluate the speedup and accuracy of *SpecEE* and baseline on MT-Bench, SUM, QA, Alpaca, GSM8K, HumanEval in Section 7.1.3.

A.3.5 Models. The artifact evaluation will use the Llama2-7B-chat [41], Llama-2-7B-Chat-AWQ [28], EAGLE-llama2-chat-7B [27]. The detailed configuration is in Section 7.1.3.

A.4 Installation

The installation is the environment setup for both cloud scenario and PC scenario. For the both scenarios, please follow the commands to set up the environment. Make sure we have the conda environment.

A.4.1 Cloud Scenario. Follow the commands below for setup the *SpecEE* environment.

```
1 cd SpecEE_cloud
2 conda create -n SpecEE python==3.10
3 conda activate SpecEE
4 pip install -r requirements.txt
```

Follow the commands below for setup the baseline AWQ [28] environment.

```
1 cd SpecEE-cloud
2 conda create -n awq python==3.10
3 conda activate awq
4 pip install -r requirements_awq.txt
```

Follow the commands below for setup the AWQ [28] integrated *SpecEE* environment.

```
1 cd SpecEE-cloud
2 conda create -n specee_awq python==3.10
3 conda activate specee_awq
4 pip install -r requirements_awq.txt
5 cd AutoAWQ-0.2.6
6 pip install -e .
```

A.4.2 PC Scenario. Follow the commands below for setup the AWQ [28] integrated *SpecEE* environment.

```
1 cd SpecEE_PC
2 conda create -n specee python=3.12
3 conda activate specee
4 pip install -r ./requirements/requirements-convert_hf_to_gguf.txt
```

A.5 Experiment workflow

Please follow the workflow to reproduce the key results on speedup in Figure 14 and Figure 16 and accuracy in Table 4 for two scenarios. The overall workflow is also provided in the README.md of two scenario file folder.

A.5.1 Cloud Scenario. (1) Prepare the large language models.

```
1 huggingface-cli login
2 huggingface-cli download meta-llama/Llama-2-7b-chat-hf
3 huggingface-cli download TheBloke/Llama-2-7B-Chat-AWQ
4 huggingface-cli download yuhui/EAGLE-llama2-chat-7B
```

(2) Evaluate the speedup and accuracy performance between the *SpecEE* and baseline Huggingface.

Firstly, we activate *SpecEE* environment.

```
1 conda activate SpecEE
```

Secondly follow the example command below to evaluate speed.

```
1 CUDA_VISIBLE_DEVICES=0 python EEInference.py --base-model-path meta-llama/Llama-2-7b-chat-hf --draft-model-path yuhui/EAGLE-llama2-chat-7B --dataset mt_bench --task speed --predictor-path [the local path of ./llama-7b]
```

We can change the parameter after **-dataset** to get the speedup on other datasets. We support the mt-bench, sum, qa, alpaca, gsm8k and humaneval datasets.

Finally we can see the speed evaluation results output.

```
1 % The speedup output example on MT_bench dataset
2 % SpecEE mt_bench tokens per second :
54.66483409619601
```

```

3 % HF mt_bench tokens per second :
4 45.520089307858285
5 % SpecEE acceleration ratio is: 1.2008947022597127

```

To evaluate the accuracy, please follow the command below.

```

1 CUDA_VISIBLE_DEVICES=0 python EEInference.py --base-
2 model-path meta-llama/Llama-2-7b-chat-hf --draft-
3 model-path yuhuili/EAGLE-llama2-chat-7B --dataset
4 sst2 --task accuracy --predictor-path [the local
5 path of ./llama-7b]

```

We can change the parameter after **-dataset** to get the speedup on other datasets. We support the sst2 and commonsenseqa datasets. Then we can see speed evaluation results.

```

1 % The accuracy output example on sst2 dataset
2 % SpecEE Model accuracy on sst2 is:
3 0.8704128440366973
4 % HF Model accuracy on sst2 is: 0.8715596330275229

```

We also provide the **faster and easier execution** method. We need to replace the model path information in **run_hf_spec_ee.sh** to complete all experiments replication.

```

1 ./run_hf_spec_ee.sh

```

(3) Evaluate the speed and accuracy performance between awq integrated SpecEE and baseline awq.

To evaluate the accuracy, please follow the command below.

```

1 conda activate SpecEE
2 CUDA_VISIBLE_DEVICES=0 python EEInference_awq.py --
3 base-model-path TheBloke/Llama-2-7B-Chat-AWQ --draft
4 -model-path yuhuili/EAGLE-llama2-chat-7B --dataset
5 commonsenseqa --task accuracy --predictor-path [the
6 local path of ./llama-7b]

```

We can change the parameter after **-dataset** to get the speedup on other datasets. We support the sst2 and commonsenseqa datasets. To evaluate the speedup, please follow the command below.

We can get all awq speed by the command below.

```

1 conda activate awq
2 CUDA_VISIBLE_DEVICES=0 python AwqInference.py --base-
3 model-path TheBloke/Llama-2-7B-Chat-AWQ

```

We can get all awq integrated *SpecEE* speed by the command below.

```

1 conda activate spec_ee_awq
2 CUDA_VISIBLE_DEVICES=0 python AwqEEInference.py --
3 base-model-path TheBloke/Llama-2-7B-Chat-AWQ --draft
4 -model-path yuhuili/EAGLE-llama2-chat-7B

```

The speed evaluation results of awq and awq integrated *SpecEE* are in **raw_awq.json** and **spec_ee_awq.json**.

We can run **calculate_awq_speedup.py** to get speedup.

```

1 python calculate_awq_speedup.py
2 % Output:
3 % Speedup ratio:
4 % mt_bench: 1.1266
5 % sum: 1.1170
6 % qa: 1.1410
7 % alpaca: 1.1362
8 % gsm8k: 1.1010
9 % humaneval: 1.1162
10 % AWQ+SpecEE Average speedup: 1.1230

```

A.5.2 PC Scenario. (1) Prepare the large language models. Please follow the first step of Cloud Scenario in Section A.5.1 to download the models. And then we need to download the special models for PC scenario.

```

1 huggingface-cli login
2 huggingface-cli download YYDH2333/SpecEE-7b-chat-hf

```

(2) Compilation.

```

1 make GGML_CUDA=1 -j$(nproc)

```

We can run the example to check the compilation.

```

1 CUDA_VISIBLE_DEVICES=0 \
2 ./llama-cli \
3 -m models/SpecEE-7b-chat-hf.gguf \
4 -p "Compose an engaging travel blog post about a
5 recent trip to Hawaii, highlighting cultural
6 experiences and must-see attractions." \
7 -e -ngl 16 -t 4 -n 256 -c 512 -s 8 --top_k 0 --temp 0

```

(3) Evaluation on Speedup.

```

1 % Evaluation on alpaca dataset
2 python eval_on_alpaca_dataset.py.
3 % Evaluation on gsm8k dataset
4 python eval_on_gsm8k_dataset.py
5 % Evaluation on humaneval dataset
6 python eval_on_humaneval_dataset.py
7 % Evaluation on mt-bench dataset
8 python eval_on_mt-bench_dataset.py
9 % Evaluation on qa dataset
10 python eval_on_qa_dataset.py
11 % Evaluation on sum dataset
12 python eval_on_sum_dataset.py

```

A.6 Evaluation and expected results

The results generated from this artifact should match those shown in result figures and tables. It is notable that the results will differ a little because small difference in the hardware and software platform especially the speedup. Small differences might be observed in other experiments as well, but they do not influence the and overall result and conclusion of this paper.

A.7 Notes

Some evaluations may take more than an hour due to the large dataset. Importantly, please refer to the ReadME.md file for more introduction on evaluation and code introduction.

A.8 Methodology

Submission, reviewing and badging methodology:

- <https://www.acm.org/publications/policies/artifact-review-and-badging-current>
- <https://cTuning.org/ae>